

Southwest
Tech

Python Programming Foundations

Week 3 - Organizing Code and Data
Day 3: "Comparing Rough & Clean Program Structures"

1

Southwest
Tech

Working Code Can Still Improve

Working code is important, but it is not the only goal.

Code can run and still be hard to read, explain, or change.

Today, compare structure by asking:

- ▶ What got clearer?
- ▶ What got easier to change?
- ▶ What became easier to explain?
- ▶ What stayed small enough to trace?

IMPROVE

This Photo by Unknown Author is licensed under CC BY

2

Southwest
Tech

Same Goal. Different Structure.

Both work. The right side is easier to read, reuse, and grow.

Rough Working Code

Repeated parts, convoluted, harder to scan

```

1 tasks = ["read", "practice", "submit", "review"]
2 scores = [10, 15, 10, 10]
3
4 print("Task Report")
5 print("-----")
6 print(f"{tasks[0]} = {scores[0]}")
7 print(f"{tasks[1]} = {scores[1]}")
8 print(f"{tasks[2]} = {scores[2]}")
9 print(f"{tasks[3]} = {scores[3]}")
10
11 total = scores[0] + scores[1] + scores[2] + scores[3]
12 avg = total / 4
13 print(f"Average Score: {avg:.1f}")
14
15 high = 0
16 if scores[0] > high: high = scores[0]
17 if scores[1] > high: high = scores[1]
18 if scores[2] > high: high = scores[2]
19 if scores[3] > high: high = scores[3]
20 print(f"Highest Score: {high}")

```

Works, but looks like a mess and change. Fixed code later, now more work to be edited.

Cleaner Organized Code

Named functions, grouped data, easier to scan

```

1 # Data: prompt and data to collect
2 tasks = ["read", "practice", "submit", "review"]
3 scores = [10, 15, 10, 10]
4
5 def print_report(tasks, scores):
6     print("Task Report")
7     print("-----")
8     for i in range(len(tasks)):
9         print(f"{tasks[i]} = {scores[i]}")
10
11 def avg_score(scores):
12     return sum(scores) / len(scores)
13
14 def highest_score(scores):
15     return max(scores)
16
17 # Main: start the program
18 print_report(tasks, scores)
19 print(f"Average Score: {avg_score(scores):.1f}")
20 print(f"Highest Score: {highest_score(scores)}")

```

More readable. Easier to read, reuse, and extend. Add new things to change the logic without modifying many lines.

Working but Improvable

3

Southwest
Tech

Functions Plus Data Structures

Functions organize logic.

Lists and dictionaries organize information.

Cleaner programs often use both:

- ▶ named jobs
- ▶ related data grouped together
- ▶ output that is easier to explain

4

Southwest
Tech

Functions Work With Data Structures

Named jobs + related data = visible output

1. Named Jobs (Functions)

Functions are reusable ideas that do a job.

```

get_tasks() Get the list of tasks
add_task(task) Add a task to the list
show_tasks(tasks) Print each task

```

2. Related Data (Data Structures)

Data is stored in structures that functions can use.

```

tasks (list)
+ read
+ practice
+ submit
new_task (string)
"review"

```

3. Visible Output (Results)

The results are shown so people can see them.

```

Task List
1. read
2. practice
3. submit
4. review

```

How It Works (Example)

```

get_tasks() → show the data (tasks list) → add_task("review") → tasks is updated (tasks list) → show_tasks(tasks) → visible output on the screen

```

Key Idea: Functions (named jobs) work with data (related data) to produce results we can see (visible output).

Functions Plus Data

5

What Counts as Success Today

Working baseline -> compare structure -> revise one meaningful thing -> explain the improvement.

A useful revision should make the program:

clearer	easier to test	easier to change	easier to explain
---------	----------------	------------------	-------------------

6

Revise in Small, Meaningful Steps
A simple 4-step flow you can repeat.

1 Working Baseline
Start with something that already works.
Example: Tests that: read, practice, submit. Prints each task.

2 Compare Structure
Look at your data and code structure.
Check: Is the structure a good match for the job? Is the code easy to read and reuse? Is anything repeated unnecessarily?

3 Revise One Meaningful Thing
Make one clear change that improves the code or the data.
Example Change: Before: Repeated prints. After: Loop to print each task. Small change, big impact.

4 Explain Improvement
Say what is better and why it matters.
My Improvement: I used a loop to print each task. Now the code is shorter, easier to read, and easier to update.

Keep It Going
Repeat these four steps often. Small improvements add up to strong, clear programs.

Today's Success

7

What We Will Use Today

- ▶ rough working code
- ▶ cleaner organized code
- ▶ function responsibility
- ▶ list/dictionary choice
- ▶ comparison questions
- ▶ small revision

The goal is better structure, not more features.

8

Revision Toolbox
Simple tools to make your code and data better, step by step.

1. Rough Baseline
Get something working. Don't worry about perfect.
Example: print("Task done"), print("... next"), print("... next"), print("... next")

2. Cleaner Version
Organize code and data as it's easier to read.
Example: tasks = ["read", "practice", "submit"], for i, task in enumerate(tasks, 1): print(f"{i}. {task}")

3. Function Responsibility
Each function does one clear job.
Examples: get_data() → get data, add_task() → add one item, show_tasks() → display items

4. Data Choice
Pick the structure that fits the job.
List (ordered): keep order, loop through, use indices. Dictionary (labeled): use keys, look up values, labels matter.

5. Comparison Questions
Ask these before you make changes.
Questions: Does the data structure fit the job? Is the code easy to read and reuse? Is anything repeated unnecessarily? Will this make future changes easier?

6. Small Revision
Make one meaningful change at a time.
Examples: Replace repeated prints with a loop, Move repeated code into a function, Separate according to the choice, Choose a better data structure.

Use these tools often. Small steps lead to big improvements. Remember: Clear code = good data choice = easy to read, reuse, and grow.

Toolbox

9

What We Will "Skip" For Now

- ▶ Broad redesign
- ▶ Class-based architecture
- ▶ Large AI rewrites
- ▶ Project Restructuring

Today's revision should be small enough to explain:

- ▶ one function improvement
- ▶ one data-structure improvement
- ▶ one clearer name
- ▶ one removed repetition

10

Parked for Later
Powerful ideas. Best used when the basics are solid.

Broad Redesign
Rethinking the overall approach or structure from the ground up.

Class-Based Architecture
Organizing code around classes and object relationships.

Large AI Rewrites
Using AI to recreate or transform large parts of the code.

Project Restructuring
Changing folders, modules, or layout on a large scale.

Save these until the current structure is clear.

Parked For Later

11

Correct Is Not The Same As Clear

Correct code produces the expected result.
Clear code helps a human understand how the result happens.

Both matter because:

- ▶ programs need to run
- ▶ people need to read them
- ▶ future changes need to be possible

12

Southwest Tech

Two Sides of Good Code

Great code is both correct and clear.

Correct

The code produces the expected output.

Expected Output

```
Task Report
-----
1. read      - 90
2. practice - 85
3. test1    - 78
4. review   - 98
-----
Average Score: 86.3
Highest Score: 98
```

- Runs without errors
- Matches the expected results
- Solves the problem correctly

Clear

The code is easy to read, understand, and explain.

Readable Structure

```
def average_score(scores):
    return sum(scores) / len(scores)

def highest_score(scores):
    return max(scores)

def print_report(tasks, scores):
    for i, (task, score) in enumerate(zip(tasks, scores)):
        print(f"{i+1}: {task} - {score}")

# Example usage
tasks = ["read", "practice", "test1", "review"]
scores = [90, 85, 78, 98]
print_report(tasks, scores)
```

Clear Explanation

- Functions do one job each
- Data is grouped and easy to follow
- Easy to change, test, and reuse
- Easy to read and follow
- Easy to explain to someone else
- Easy to change and build on

Both matter equally.
Code should be correct in what it does and clear in how it does it.

Correct vs. Clear

13

Southwest Tech

Cleaner Structure Is Easier To Explain

Cleaner structure usually makes the program easier to talk through - Look for:

- ▶ clear function names
- ▶ related data grouped together
- ▶ less repeated logic
- ▶ output that matches the program's purpose

If you can explain it better, the structure probably improved.

14

Southwest Tech

Clear Functions + Grouped Data = Understanding

Good names. Good groups. Easy to explain.

Clear Function Names

Each function does one clear job.

```
get_tasks()
add_tasks(tasks)
calculate_total_score()
show_summary(tasks, scores)
```

Grouped Data

Related data lives together.

```
tasks = [
    "read", "practice",
    "test1", "review"
]

scores = [
    90, 85, 78, 98
]

settings = {
    "printing": True,
    "show_details": True
}
```

Works Together

Data flows through clear steps.

```
get_tasks()
↓
add_tasks(tasks)
↓
calculate_total_score()
↓
show_summary(tasks, scores)
```

I can explain this.

Clear names: Make the program obvious.
Grouped data: Keep related things together.
Easy to explain: You and others understand the flow.

Explainable Structure

15

Southwest Tech

Comparison Questions

Use comparison questions instead of vague opinions.

Ask:

- ▶ What changed?
- ▶ Why does it help?
- ▶ What became easier?
- ▶ What stayed the same?
- ▶ What would I change next?

16

Southwest Tech

Quick Review Checklist

Use these five questions to review any revision.

1	What changed? Identify the specific change you made.	
2	Why does it help? Explain how this change improves the code or data.	
3	What became easier? Describe what is easier to do now.	
4	What stayed the same? List what did not change (and why that's okay).	
5	What would I change next? Note one small improvement to try next.	

Comparison Questions

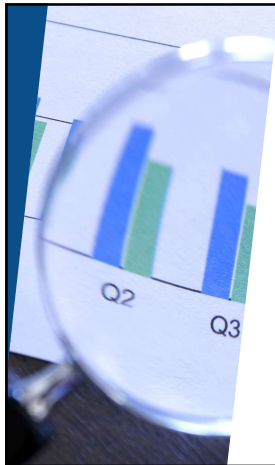
17

Southwest Tech

Demo #1

"Rough vs Organized"

18



What to Watch For

Watch both versions run.

Then compare:

- ▶ repeated code
- ▶ function responsibility
- ▶ name clarity
- ▶ how easy each version is to change

19



What Changed?

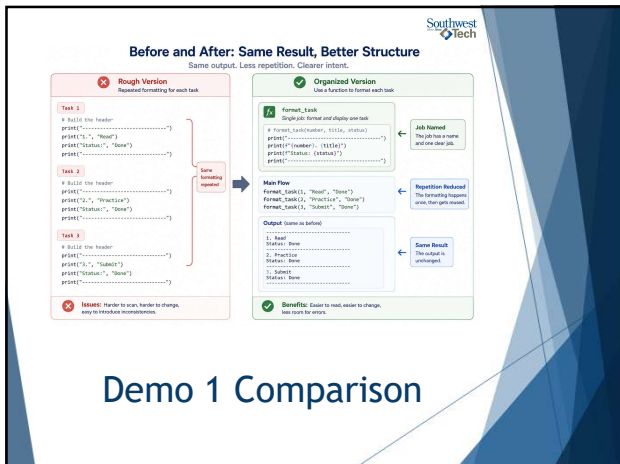
A useful comparison names specific changes.

Examples:

- ▶ repeated output formatting moved into a function
- ▶ status logic received a clear name
- ▶ task data became easier to reuse
- ▶ one change affects multiple outputs

Specific changes are easier to justify than general impressions.

20



Before and After: Same Result, Better Structure

Same output. Less repetition. Clearer intent.

Rough Version
Repeated formatting for each task

Organized Version
One function to format each task

Task 1
Build the header
print("-----")
print(f"Task 1: {task1}")
print("-----")

Task 2
Build the header
print("-----")
print(f"Task 2: {task2}")
print("-----")

Task 3
Build the header
print("-----")
print(f"Task 3: {task3}")
print("-----")

Issues: Harder to scan, harder to change, easy to introduce inconsistencies

Format_Task
Single job: format and display one task

Main Flow
format_task(1, "read", "Done")
format_task(2, "test", "Done")
format_task(3, "change", "Done")
format_task(4, "reuse", "Done")
format_task(5, "explain", "Done")

Repetition Reduced
The formatting happens once, then gets reused

Same Result
The output is unchanged

Output (Same as before)
1. read
Status: Done
2. test
Status: Done
3. change
Status: Done
4. reuse
Status: Done
5. explain
Status: Done

Benefits: Easier to read, easier to change, easy code for others

Demo 1 Comparison

21



What Got Easier?

An improvement should make something easier.

Ask whether the revision made it easier to:

- ▶ read
- ▶ test
- ▶ change
- ▶ reuse
- ▶ explain

If nothing got easier, the revision may not be useful yet.

22



Good Improvements Make Work Easier

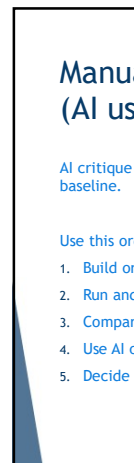
Any change is worth it if it makes at least one of these easier.

- read**
Understand the code quickly without confusion.
- test**
Check behavior and catch issues with confidence.
- change**
Make updates without breaking other parts.
- reuse**
Use the code or logic again in other places.
- explain**
Describe how it works to others clearly.

★ **Aim for progress.**
If your change makes at least one of these easier, you're on the right track.

What Got Easier

23



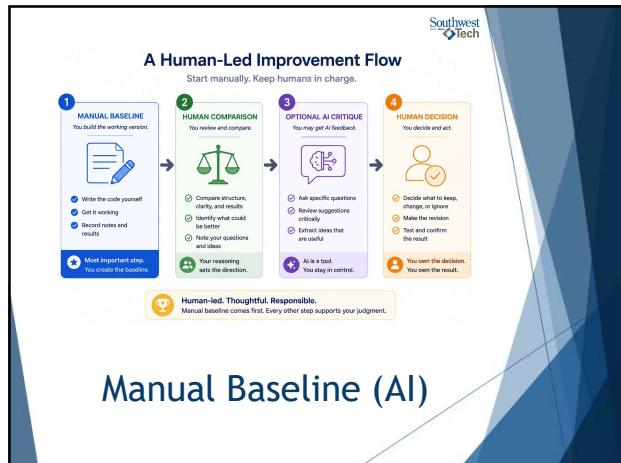
Manual Baseline Comes First (AI use)

AI critique only helps learning after you have your own baseline.

Use this order:

1. Build or revise manually.
2. Run and inspect your version.
3. Compare structure yourself.
4. Use AI only if permitted.
5. Decide what to keep, change, or reject.

24



25

AI Suggestions Are Not Automatic Answers

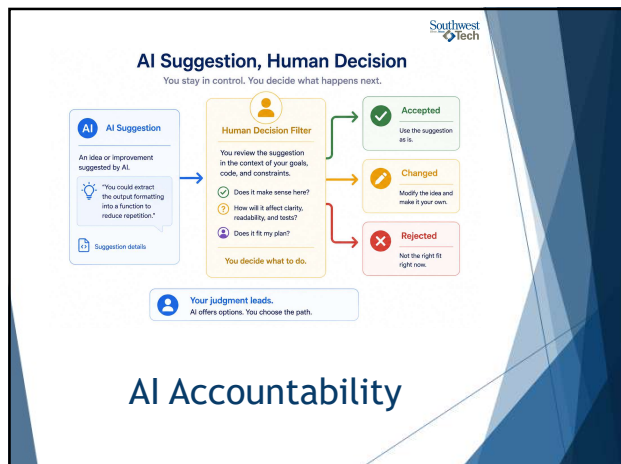
If AI suggests a refactor, you still decide whether it fits.

Explain:

- ▶ what AI suggested
- ▶ what you accepted
- ▶ what you changed
- ▶ what you rejected
- ▶ why the final version makes sense

AI can suggest structure. You must understand and justify it

26



27

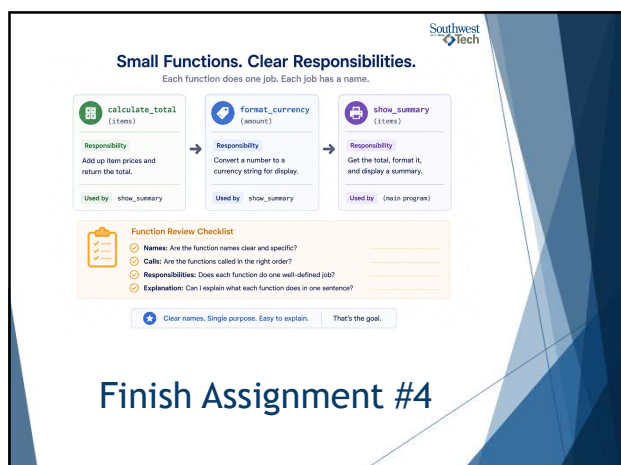
Finish Assignment #4

Check that your function structure is explainable.

Your program should have:

- ▶ at least two functions
- ▶ meaningful function names
- ▶ correct function calls
- ▶ one clear responsibility per function
- ▶ explanation of how the functions work together

28



29

Finish Assignment #5

Check that your data structure is useful.

Your program should have:

- ▶ list or dictionary data
- ▶ a lookup, retrieval, or loop
- ▶ visible output
- ▶ a structure choice you can explain
- ▶ code that stays small enough to trace

30

Southwest
Tech

Data Mini-App: From Structure to Output

Good structure makes the output easy to produce and easy to explain.

1. Data (Structure Choice)

We'll use a list of dictionaries to store student scores.

```
students = [
    {'name': 'Helen', 'score': 82},
    {'name': 'Jordan', 'score': 88},
    {'name': 'Lucia', 'score': 78}
]
```

Why this structure?
Each student has two related pieces of data (name and score), and it keeps the students in order.

2. Mini-App Logic

Loop through the list and build the output lines.

- Do through each student
- Create a line with name and score
- Collect the lines

3. Visible Output

Clear, readable result from our data.

```
Student Scores
Helen 82
Jordan 88
Lucia 78
```

Structure Choice Explained

A list of dictionaries fits this problem because:

- Each student has multiple related values (name and score).
- We need to keep the students in order.
- It's easy to loop through and format for display.

Good data structure → Simple logic → Clear output → Easy explanation

Finish Assignment #5

31

Southwest
Tech

Revision Evidence

It should explain what improved.

Useful explanation:

- ▶ what changed
- ▶ why it changed
- ▶ how the new version is clearer
- ▶ what you tested after the change
- ▶ what you would improve next

32

Southwest
Tech

Evidence for Week 3



Should show organization and explanation.

Useful evidence includes:

- ▶ working ".py" files
- ▶ function responsibility explanation
- ▶ data structure choice explanation
- ▶ visible output
- ▶ AI-use note if AI was permitted and used

The goal is not just code that runs. The goal is code you can explain.

33

Southwest
Tech

My Mini-App Evidence

Code + Output + Explanation = Understanding

1. My .py Files

These files work together:

- main.py: Runs the program and shows output
- tasks.py: Functions for task operations
- data.py: Holds the task data structure

4. Output Sample

This is what my program prints.

Task List

```
1. Read Chapter 3 [done]
2. Write summary [in progress]
3. Practice problems [todo]
```

2. Function Explanation

Each function has a clear job.

```
get_tasks() Returns the list of tasks.
format_tasks() Turns one task into a nicely formatted string.
show_tasks(tasks) Loops through tasks and prints each one.
```

3. Data Structure Choice

I choose a list of dictionaries.

```
tasks = [
    {'title': 'Read Chapter 3',
     'status': 'done'},
    {'title': 'Write summary',
     'status': 'in progress'},
    {'title': 'Practice problems',
     'status': 'todo'}
]
```

Why this structure?
Each task has multiple pieces of data, and a list keeps them in order.

What I Learned

- I used functions to organize the work.
- I used a list of dictionaries to store related data.
- I can explain how my program works from data to output.

Evidence Checklist .py files included Functions explained Data structure explained Output shown

Evidence

34

Southwest
Tech

Closing Success Check

If you can justify the improvement, you understand it. Before submitting Week 3 work, ask:

- ▶ What did I organize?
- ▶ What got clearer?
- ▶ What became easier to change?
- ▶ What evidence shows it still works?
- ▶ Can I explain the final structure?

35

Southwest
Tech



Thank you!

Have Fun!

36